

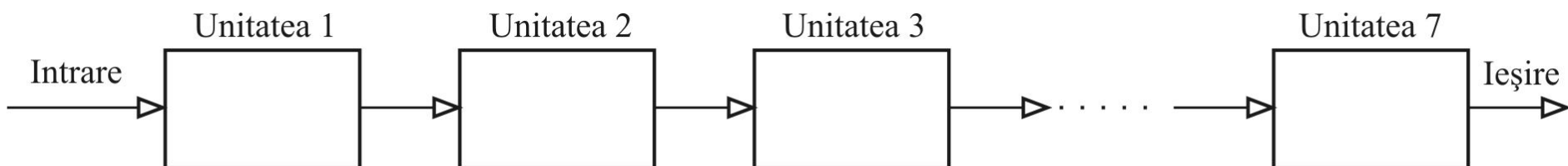
Proiectarea arhitecturilor *pipeline*

1. Concept și mod de implementare

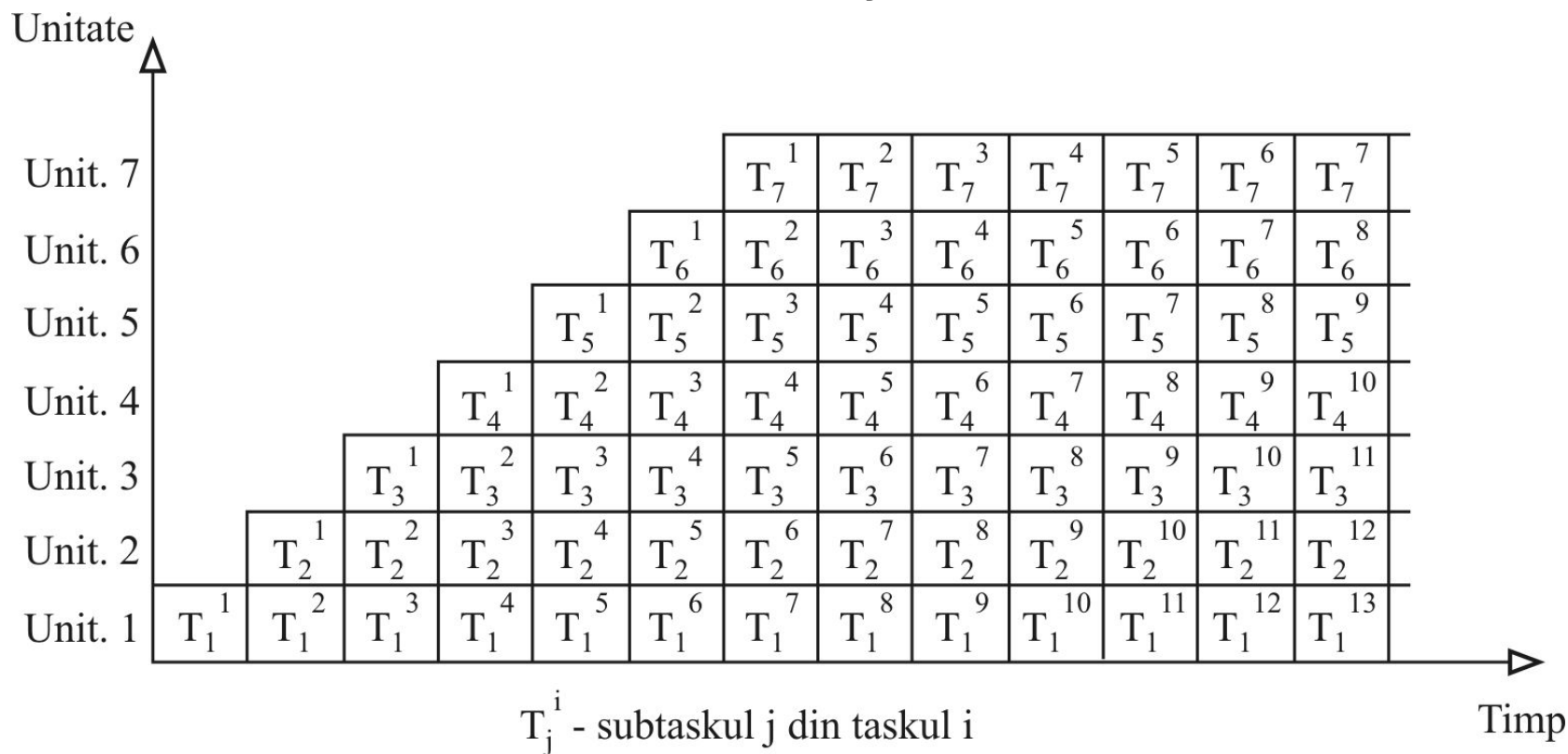
Procesarea *pipeline* reprezintă tehnica prin care un *task* este împărțit (o operație complexă este împărțită) într-un număr de *subtask*-uri (suboperații) care vor fi procesate secvențial. Fiecare *subtask* va fi executat într-o unitate *hardware* proprie. Toate aceste unități sunt conectate în cascadă, astfel încât ieșirea unei unități este conectată la intrarea unității următoare și toate unitățile operează simultan dar pentru *subtask*-uri (suboperații) diferite.

Proiectarea arhitecturilor *pipeline*

1. Concept și mod de implementare



a) Structura *pipeline* cu unități conectate în cascadă

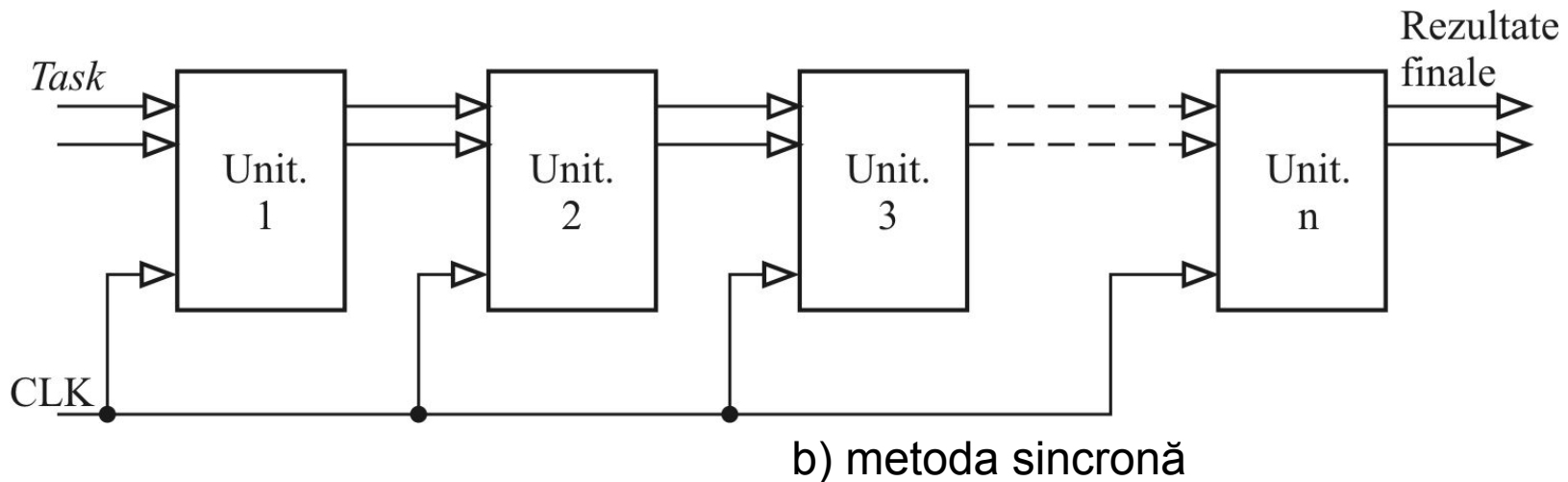
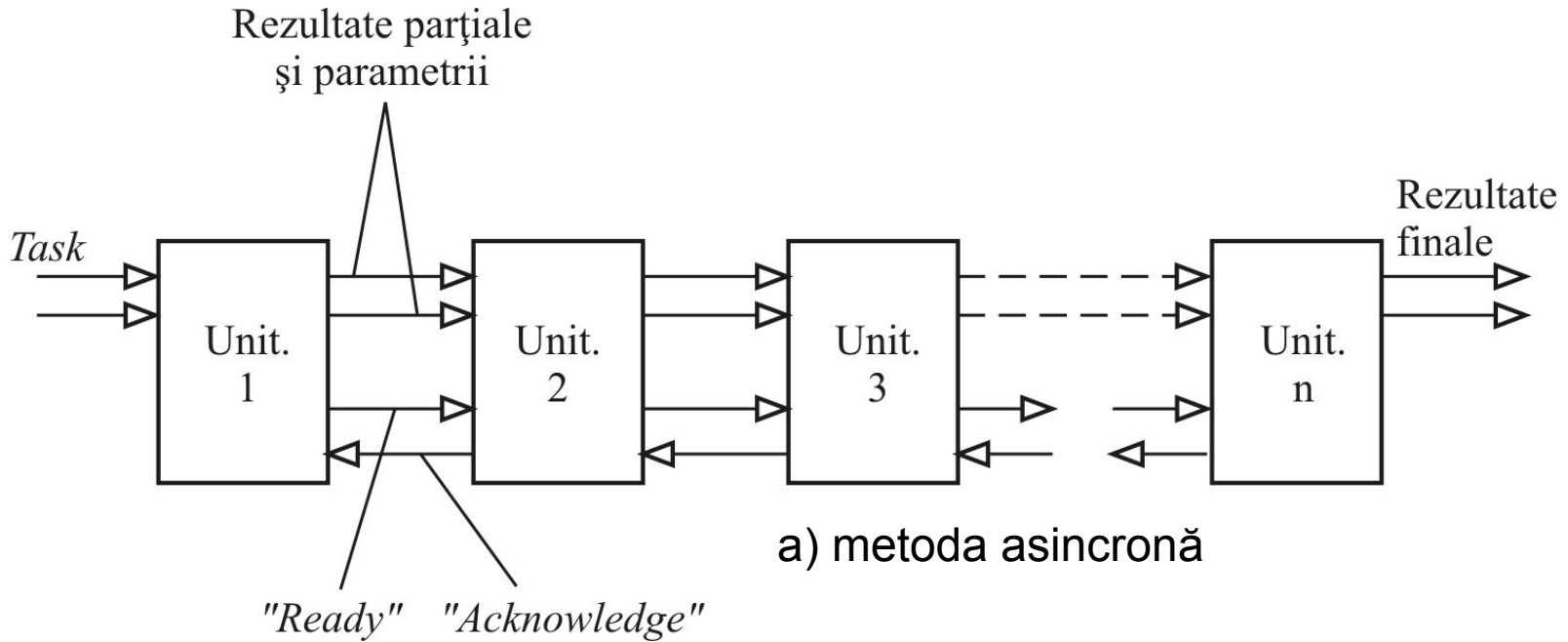


b) diagrama spațiu-timp

Procesarea *pipeline* (exemplu de structură cu 7 unități)

Proiectarea arhitecturilor *pipeline*

2. Transferul datelor în structura *pipeline*

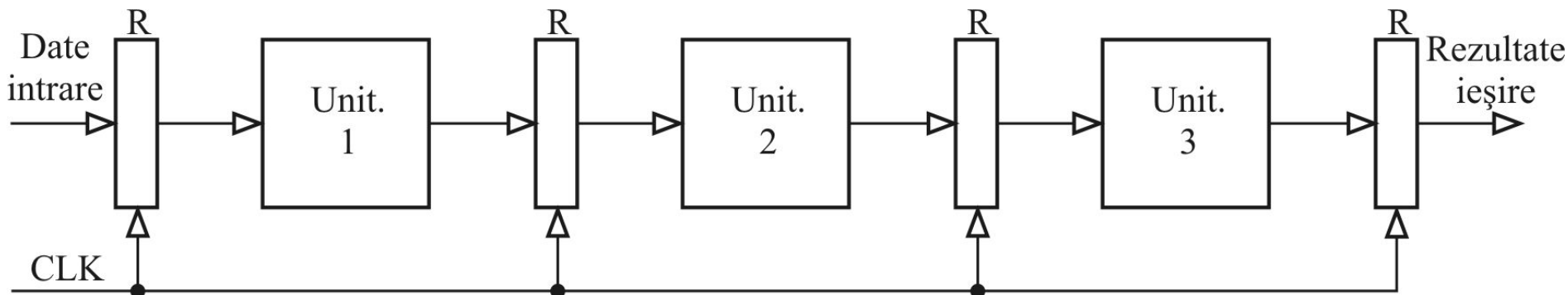


Proiectarea arhitecturilor *pipeline*

2. Transferul datelor în structura *pipeline*

Metoda asincronă permite structurii să opereze la viteză maximă, permite unităților să aibă timpi de procesare diferiți pentru valori diferite ale operanzilor de pe intrări, ceea ce conduce la o optimizare globală.

Metoda sincronă este preferată deoarece simplifică problemele de proiectare revendicând o logică de control mai simplă; în acest caz va exista câte un registru (*latch*) între fiecare două unități adiacente:



Pipeline sincron cu registre (R) între unități

Proiectarea arhitecturilor *pipeline*

3. Cost și performanțe

Dacă s *task*-uri se aplică pe intrarea unei structuri *pipeline* cu n unități, atunci vor fi necesare n perioade de tact T_{CLK} până la obținerea primului rezultat, și alte $s-1$ perioade T_{CLK} până la obținerea tuturor rezultatelor (procesarea completă a tuturor celor s *task*-uri). Deci, numărul total de perioade T_{CLK} necesare va fi $n+(s-1)$, deși fiecare *task* va fi procesat în n perioade T_{CLK} (latența structurii este n). Este de așteptat ca n perioade T_{CLK} să fie și timpul de procesare a unui *task* într-o unitate clasică, cu implementarea internă *non-pipeline*. Pentru procesarea celor s *task*-uri unitatea clasică *non-pipeline* va avea nevoie de $s \cdot n$ perioade T_{CLK} .

- **Accelerarea (S -speedup)** introdusă de structura *pipeline* comparativ cu structura *non-pipeline* va fi:

$$S = \frac{T_{non-pipe}}{T_{pipe}} = \frac{s \cdot n}{n + (s - 1)}$$

Accelerarea maximă $S=n$, se atinge pentru $s \rightarrow \infty$ și procesare fără pauze în structura *pipeline*.
Pentru $s=1$ se obține accelerarea minimă ($S=1$).

- **Eficiența (E -efficiency)** structurii *pipeline* este redată de raportul dintre accelerarea reală (obținută) raportată la acelerarea ideală (maximul obtenabil):

$$E = \frac{S}{n} = \frac{s}{n + s - 1}$$

E va atinge valoarea maximă, 1, când $s \rightarrow \infty$.
Pentru $s = 1$, E va atinge valoarea minimă $1/n$.

Proiectarea arhitecturilor *pipeline*

3. Cost și performanțe

- **Productivitatea (*H-throughput*)** structuri *pipeline*, denumită și lărgime de bandă (*bandwidth*), se definește prin numărul de *task*-uri pe care structura le poate procesa în unitatea de timp:

$$H = \frac{s}{T_{pipe}} = \frac{s}{(n + s - 1) \cdot T_{CLK}}$$

Când $s \rightarrow \infty$, H atinge valoarea maximă de un *task* (o operație) per o perioadă de tact T_{CLK}

Conceptul procesării în tehnică *pipeline* poate fi aplicat la două nivele:

- nivelul instrucțiunii : ciclul instrucțiunii este împărțit în subcicluri (faze) care se vor executa în unități specializate. Aceste unități conectate în cascadă vor forma structura *pipeline* la nivelul instrucțiunii.
- nivelul operațiilor aritmetice : operațiile aritmetice complexe (cum ar fi înmulțirea sau împărțirea în virgulă fixă sau virgulă flotantă) pot fi împărțite într-un număr de suboperații care vor fi implementate într-o structură *pipeline*.

Proiectarea arhitecturilor *pipeline*

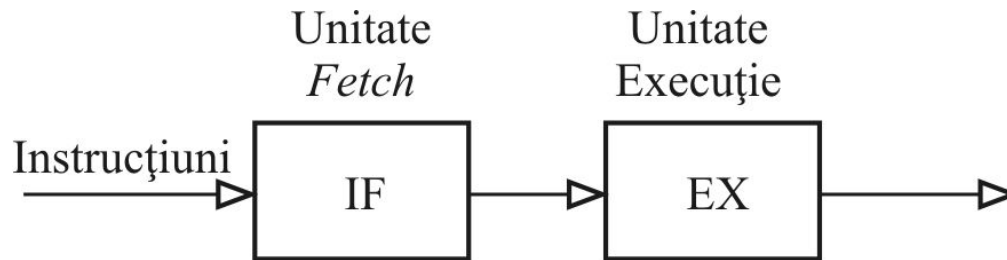
4. Structuri *pipeline* la nivelul instrucțiunii

A. Suprapunerea fazelor de *fetch* și de execuție ale instrucțiunii:

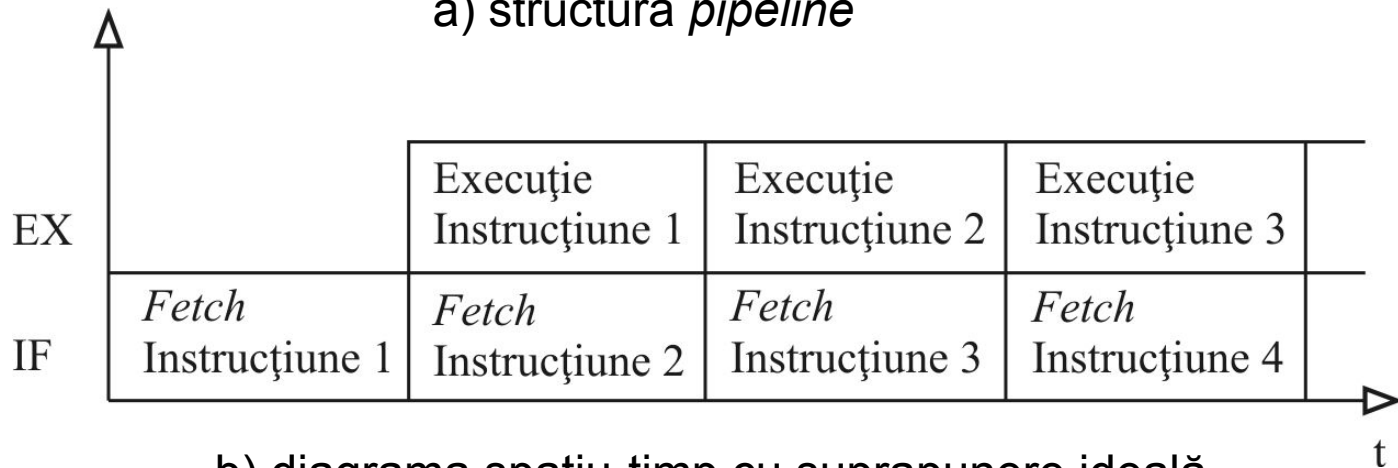
Procesarea unei instrucțiuni implică parcurgerea unei secvențe de două faze majore:

- *fetch*-ul instrucțiunii
- execuția instrucțiunii

Cele două operații pot fi efectuate într-o structură *pipeline* cu două unități:



a) structura *pipeline*



b) diagrama spațiu-timp cu suprapunere ideală

Structură *pipeline* cu două unități (*pipeline* pe două nivele)

Proiectarea arhitecturilor *pipeline*

4. Structuri *pipeline* la nivelul instrucțiunii

A. Suprapunerea fazelor de *fetch* și de execuție ale instrucțiunii:

Dacă aplicăm acest concept procesorului RISC introdus în capitolul 2, vom obține: următoarea repartizare:

- Pașii repartizați unității IF vor fi:

$MAR \leftarrow PC$

Read (citire din memorie)

$IR \leftarrow MDR ; PC \leftarrow PC + 4$

Prin urmare, registrele PC, MAR și MDR, cu logica de control aferenta, vor fi plasate în unitatea IF

- Pașii repartizați unității EX, pentru o instrucțiune R-R-R, vor fi:

Decodificare *OPCODE* ; $A \leftarrow Rs1 ; B \leftarrow Rs2$

$C \leftarrow A <\text{operație}> B$

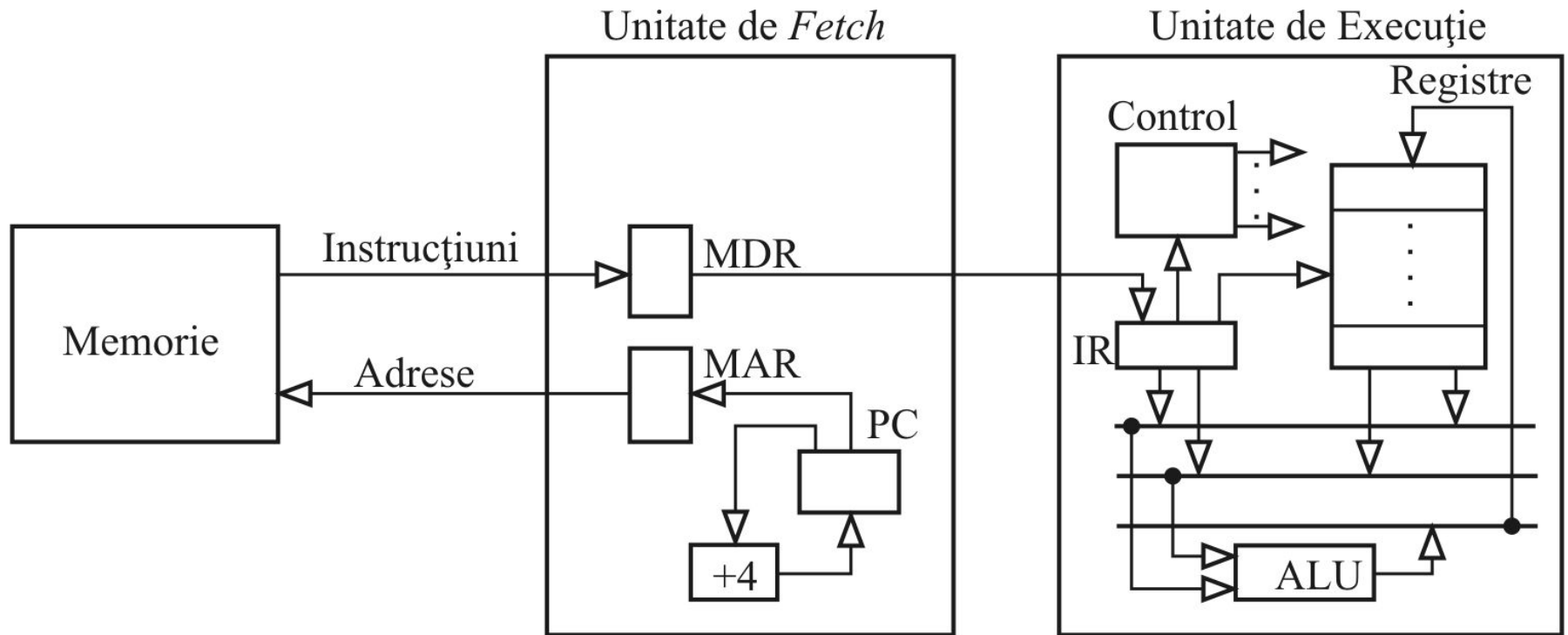
$Rd \leftarrow C$

Prin urmare, registrele generale, registrele tampon (A, B și C), unitatea ALU cu cele trei busuri și toată logica de control aferentă, vor fi plasate în unitatea EX

Proiectarea arhitecturilor *pipeline*

4. Structuri *pipeline* la nivelul instrucțiunii

A. Suprapunerea fazelor de *fetch* și de execuție ale instrucțiunii:

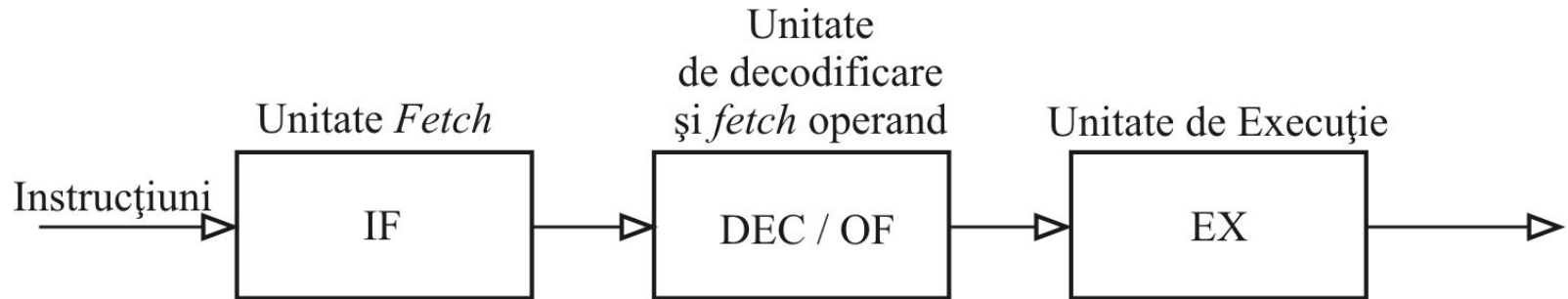


Structura *pipeline* pe două nivele aferentă procesorului RISC definit în capitolul 2

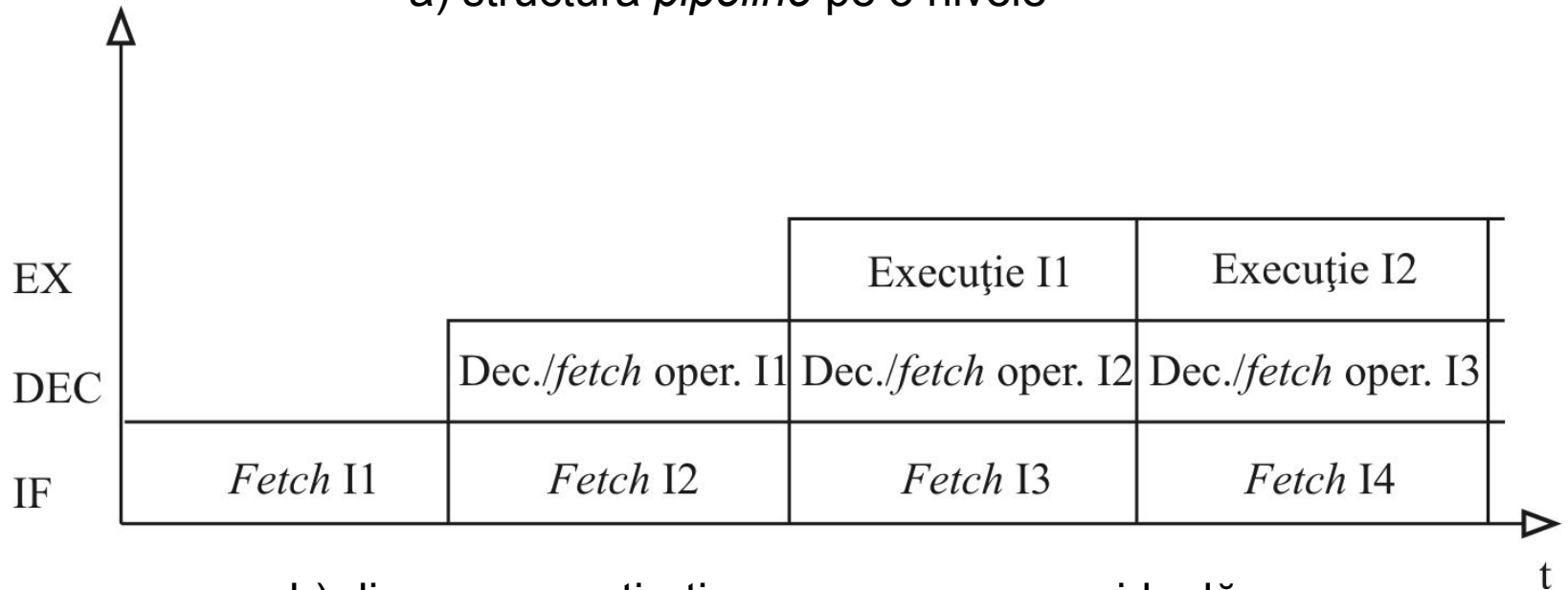
Proiectarea arhitecturilor *pipeline*

4. Structuri *pipeline* la nivelul instrucțiunii

B. Structura *pipeline* pe trei nivele:



a) structura *pipeline* pe 3 nivele



b) diagrama spațiu-timp cu suprapunere ideală

Structura *pipeline* pe trei nivele aferentă procesorului RISC definit in capitolul 2

Proiectarea arhitecturilor *pipeline*

4. Structuri *pipeline* la nivelul instrucțiunii

B. Structura *pipeline* pe trei nivele:

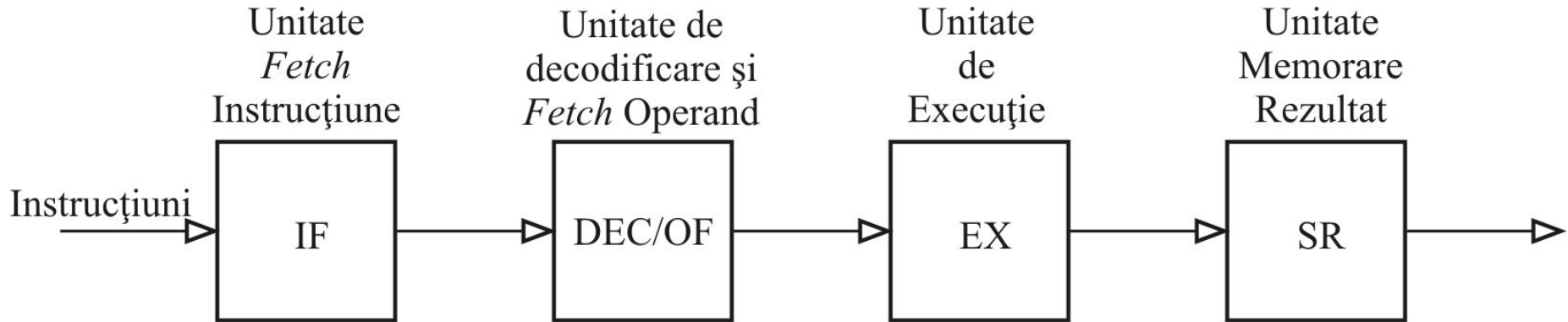
Pentru procesorul RISC definit în capitolul 2:

- Pașii repartizați unității **IF** vor fi cei definiți anterior, la structura pipeline pe două nivele (unitatea IF rămâne nemodificată):
MAR $\leftarrow$ PC
Read (citire din memorie)
IR $\leftarrow$ MDR ; PC $\leftarrow$ PC + 4
- Unitatea **DEC/OF** va decodifica instrucțiunea și va realiza *fetch*-ul operanzilor :
Decodificare *OPCODE* ; A $\leftarrow$ Rs1 ; B $\leftarrow$ Rs2
- Unitatea de **EX** va executa operația propriu-zisă și va memora rezultatul la destinație:
C $\leftarrow$ A <operație> B
Rd $\leftarrow$ C

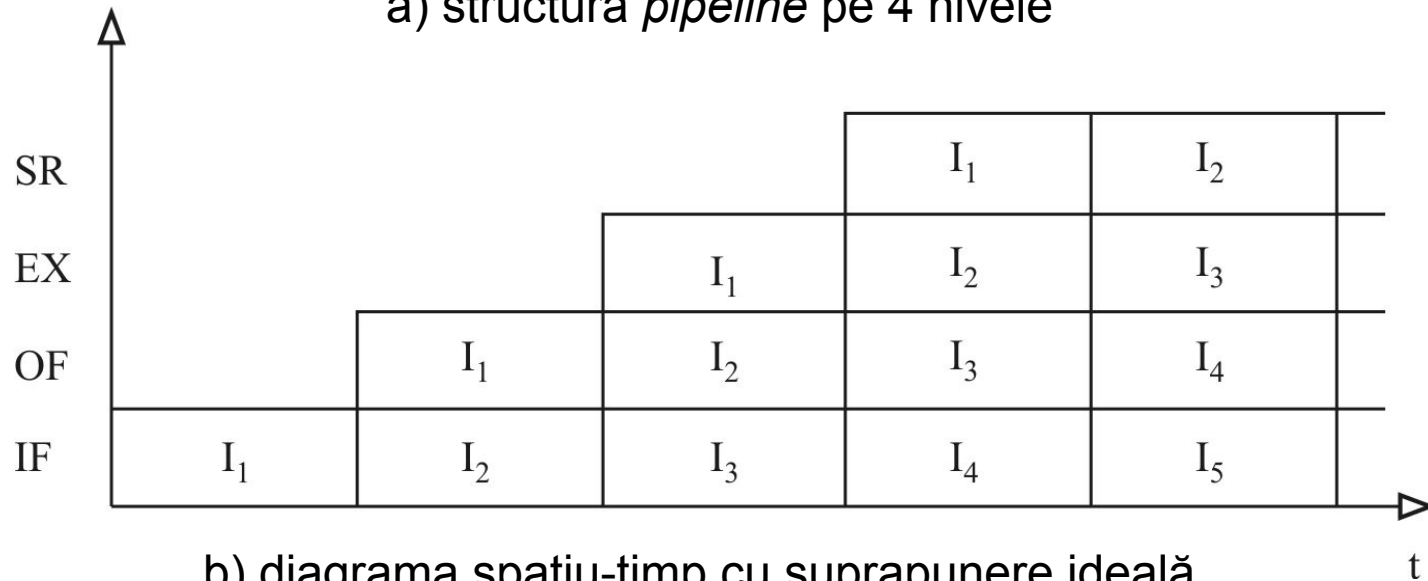
Proiectarea arhitecturilor *pipeline*

4. Structuri *pipeline* la nivelul instrucțiunii

C. Structura *pipeline* pe patru nivele:



a) structura *pipeline* pe 4 nivele



b) diagrama spațiu-timp cu suprapunere ideală
(I_i – instrucțiunea numărul i)

Structura *pipeline* pe patru nivele (utilizată de regulă la procesoarele RISC)

Proiectarea arhitecturilor *pipeline*

4. Structuri *pipeline* la nivelul instrucțiunii

C. Structura *pipeline* pe patru nivele:

Pentru procesorul RISC definit în capitolul 2:

- Pașii repartizați unității **IF** vor fi cei definiți anterior, la structura *pipeline* pe două nivele (unitatea IF rămâne nemodificată):

$MAR \leftarrow PC$

Read (citire din memorie)

$IR \leftarrow MDR ; PC \leftarrow PC + 4$

- Unitatea **DEC/OF** va decodifica instrucțiunea și va realiza *fetch*-ul operanzilor :

Decodificare *OPCODE* ; $A \leftarrow Rs1$; $B \leftarrow Rs2$

- Unitatea de **EX** va executa operația revendicată de instrucțiune:

$C \leftarrow A <\text{operație}> B$

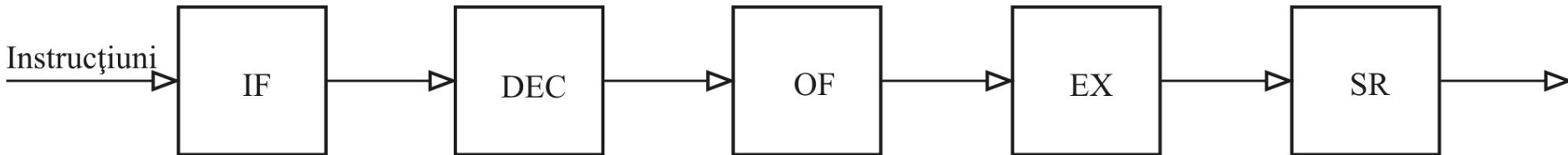
- Unitatea de **SR** (*store result*), denumita uneori si **WB**-*write back*, va memora rezultatul la destinație:

$Rd \leftarrow C$

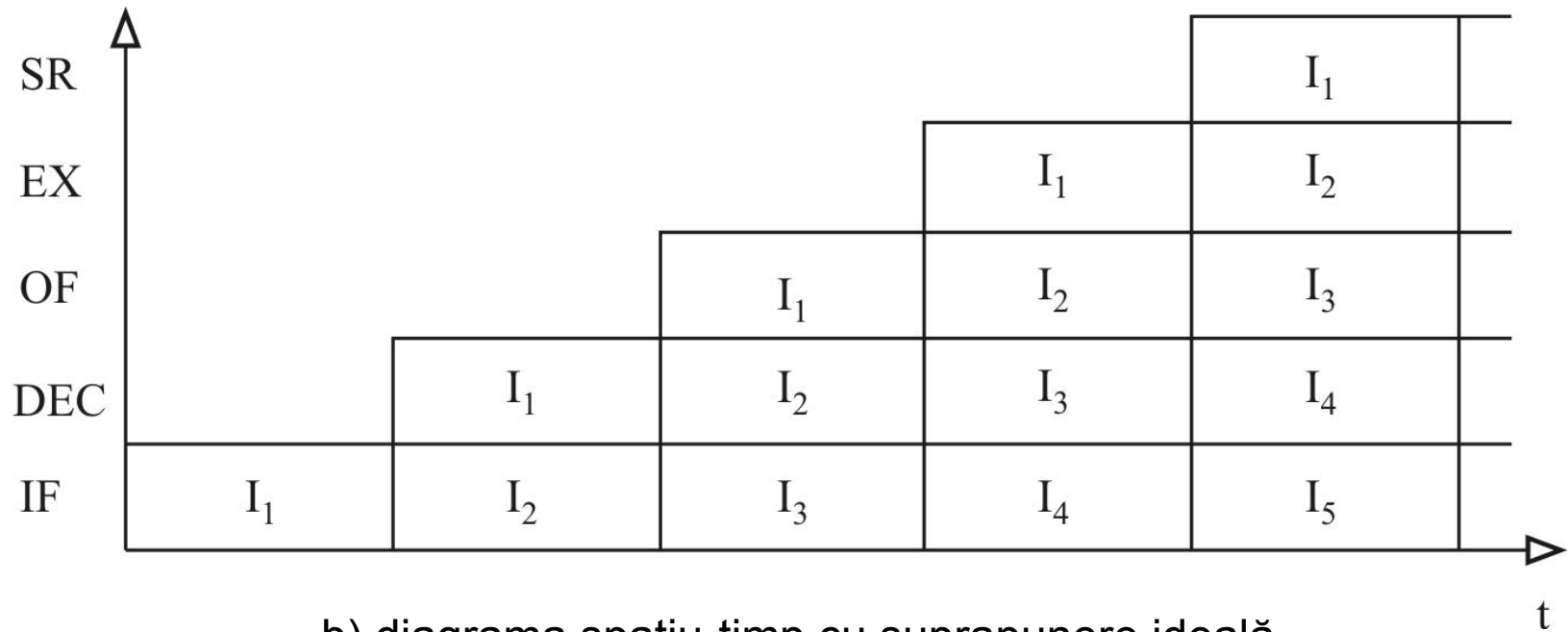
Proiectarea arhitecturilor *pipeline*

4. Structuri *pipeline* la nivelul instrucțiunii

D. Structura *pipeline* pe cinci nivele:



a) structura *pipeline* pe 5 nivele



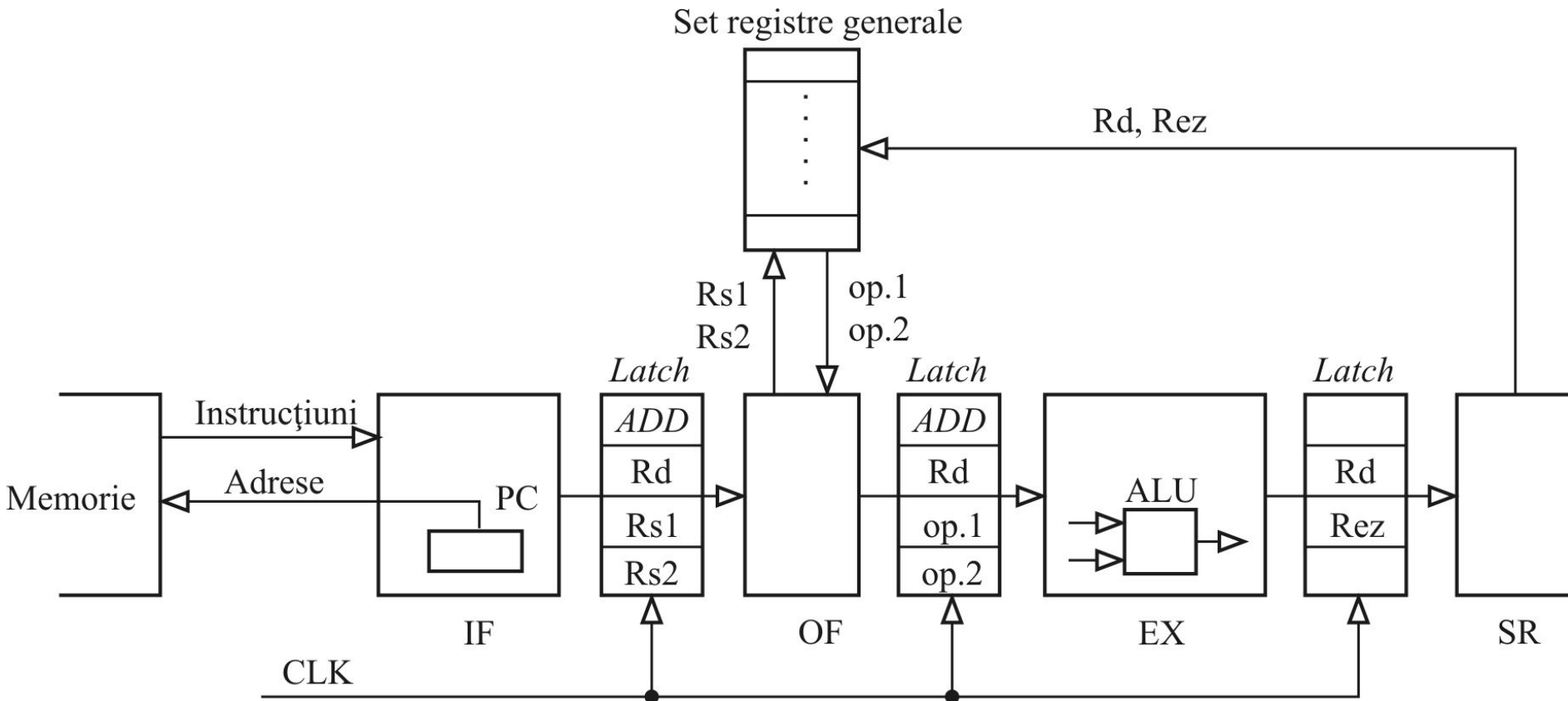
b) diagrama spațiu-timp cu suprapunere ideală

Structura *pipeline* pe 5 nivele (utilizată de regulă la procesoarele CISC)

Proiectarea arhitecturilor *pipeline*

4. Structuri *pipeline* la nivelul instrucțiunii

E. O structură *pipeline* potrivită pentru procesorul nostru RISC :

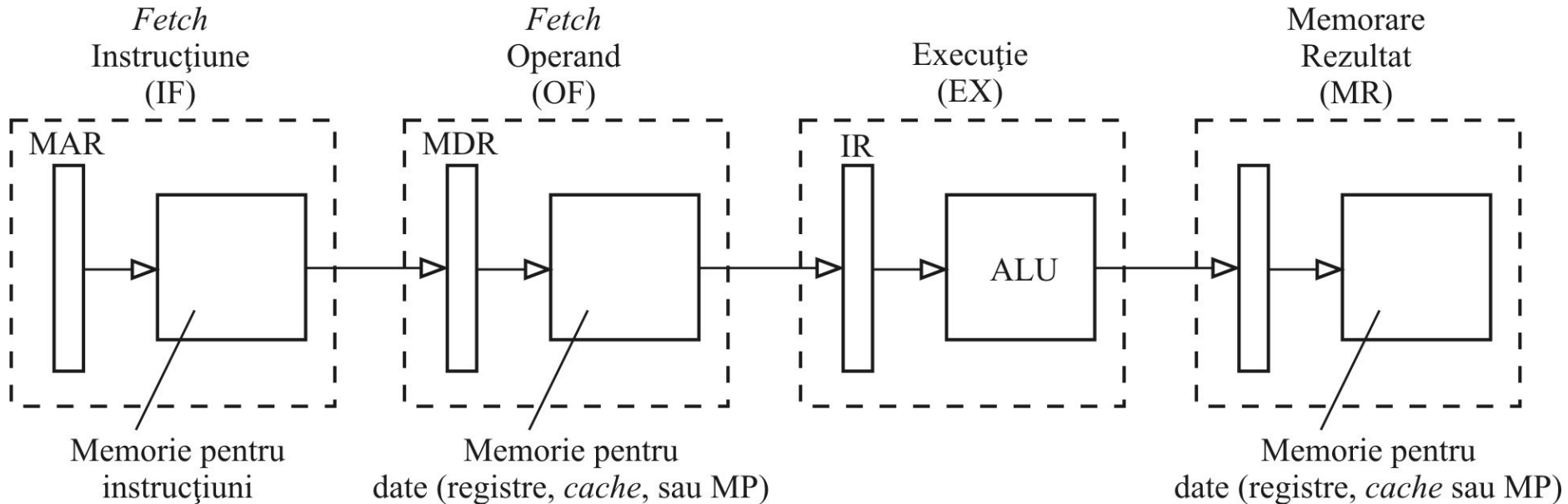


Progresul unei instrucțiuni R-R-R în structura *pipeline* pe 4 nivele

Proiectarea arhitecturilor *pipeline*

4. Structuri *pipeline* la nivelul instrucțiunii

E. O structură *pipeline* potrivită pentru procesorul nostru RISC :



O reprezentare simbolică a structurii *pipeline* care evidențiază hazardurile structurale (conflictele) la nivelul memoriei

Proiectarea arhitecturilor *pipeline*

4. Structuri *pipeline* la nivelul instrucțiunii

E. O structură *pipeline* potrivită pentru procesorul nostru RISC :

Procesorul RISC este o mașină *load/store*; execută accese la memorie dar numai în cadrul instrucțiunilor *load* (citire memorie) și respectiv *store* (scriere memorie). Ar fi oportun să fie dotat cu o singură unitate *pipeline* pentru accesarea memoriei.

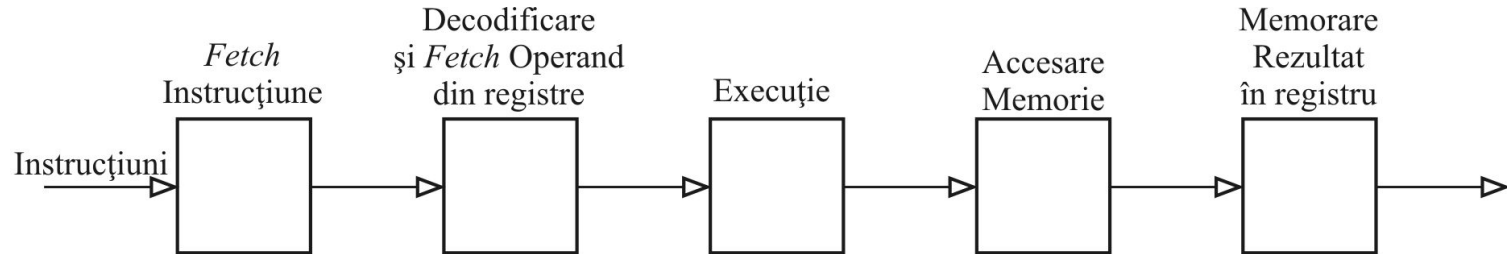
Pentru eliminarea hazardurilor structurale de la nivelul memoriei de date (MP sau *cache*) ar fi oportună următoarea organizare a unităților *pipeline*:

1. *fetch* instrucțiune.
2. decodificare instrucțiune / *fetch* operanzi din registre.
3. execuție (execută operația ALU sau calculează adresa efectivă de acces la memorie).
4. accesare memorie pentru scriere rezultat (la *store*) sau citire operand din memorie (la *load*).
5. *write back* sau *store result* (scriere rezultat în registru).

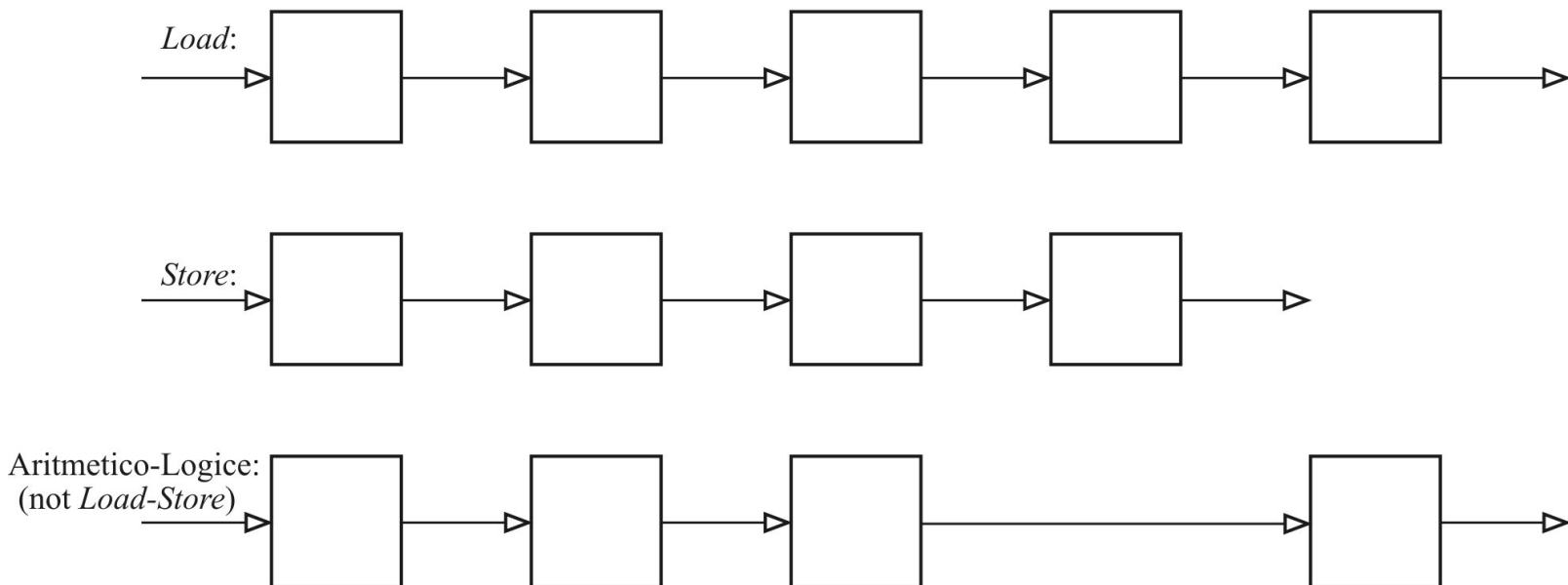
Proiectarea arhitecturilor *pipeline*

4. Structuri *pipeline* la nivelul instrucțiunii

E. O structură *pipeline* potrivită pentru procesorul nostru RISC :



a) Structura *pipeline*



b) utilizarea unităților *pipeline* în cadrul diverselor instrucțiuni

Structură *pipeline* pe 5 nivele dar cu un singur nivel pentru accesarea memoriei